

VOLUME 5

COMPLETE VOLUME — CHAPTERS 5.1–5.15

Recording Engine

The Heart of the App — From Camera Frame to Confirmed, Safe Upload

Document Title	Volume 5 — Recording Engine (Combined)
Volume Reference	Volume 5
Chapters Included	5.1 through 5.15 (15 chapters)
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Governed By	Volume 0 — Project Foundation; grounded in Volumes 1, 3, and 4

Table of Contents

This volume is the recording-and-upload engine itself: camera capture, the automatic chunking lifecycle, metadata generation, local storage, the upload queue and pipeline, background upload on both platforms, offline behavior, retry strategy, the deterministic file naming pattern, and storage cleanup — every mechanism behind the product's core promise.

Chapter	Title
5.1	Camera Module
5.2	Camera Specification
5.3	Recording Lifecycle
5.4	Recording Pipeline
5.5	Video Processing
5.6	Video Chunking
5.7	Metadata Generation
5.8	Local Storage
5.9	Upload Queue
5.10	Upload Pipeline
5.11	Background Upload
5.12	Offline Mode
5.13	Retry Strategy
5.14	File Naming Strategy
5.15	Storage Cleanup

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.1

Camera Module

The Platform Service Wrapping Wide-Angle Rear Capture

Document Title	Camera Module
Chapter Reference	5.1
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	BR-01, BR-02 (Volume 1); CameraService (Volume 3, Chapter 3.4)

1. Responsibilities

- Initialize the device's rear-facing camera only — the front camera is never opened for task recording (BR-01).
- Select and lock the wide-angle lens at exactly 0.5x or 0.6x zoom factor (BR-02) — no other zoom level is ever exposed to the recording pipeline.
- Render the live preview shown full-bleed on the Recording Screen (C-09, Chapter 2.7).
- Hand off encoded frames to the Recording Pipeline (Chapter 5.4) — the Camera Module itself does not write files or manage chunk boundaries.

2. Wide-Angle Lens Selection — Device Capability Ladder

Volume 3 (Chapter 3.1) flagged this as an open engineering risk: not every Android device exposes a dedicated ultra-wide sensor through a simple API toggle. The Camera Module resolves it with a fallback ladder, tried in order at initialization:

Tier	Approach	Result
1	Query available physical camera IDs for a lens with the required field-of-view/focal-length metadata; select it directly.	True optical wide-angle — preferred, matches BR-02 exactly.
2	If no dedicated ultra-wide sensor exists, use the primary rear sensor's own zoom-out capability down to 0.5x/0.6x, if the device reports support for it.	Optical or hybrid wide-angle at the correct factor.
3	If neither is available, block recording entirely with a Checklist-style named failure ("This device doesn't support the required wide-angle capture") rather than silently recording at the wrong field of view.	Consistent, honest failure — never a silent BR-02 violation.

3. Interaction With the Checklist

Camera/Microphone permission (BR-03): the Camera Module never attempts to initialize without both permissions already confirmed granted by the Pre-Recording Checklist (FR-CHK-01, Chapter 2.7's C-07) — permission failure is the Checklist's concern, not re-checked redundantly here.

Checklist gate (BR-04): the Recording Lifecycle (Chapter 5.3) never transitions into an active camera session unless the full Checklist has already passed.

4. What This Chapter Defers

- Exact resolution/frame rate/bitrate/codec values — Chapter 5.2 (Camera Specification).
- What happens to camera frames once captured — Chapter 5.4 (Recording Pipeline).

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.2

Camera Specification

The Exact Capture Parameters Every Chunk Is Recorded At

Document Title	Camera Specification
Chapter Reference	5.2
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Feeds	FR-META-03 (Volume 1); the capture object in Volume 4, Chapter 4.5's metadata JSON.

1. Fixed Capture Parameters

These are the values every chunk's metadata.capture block (Volume 4, Chapter 4.5) records, and the exact numbers used as the worked example there — this chapter is their authoritative source.

Parameter	Value	Rationale
Resolution	1920×1080 (Full HD)	Balances footage usability against chunk file size / upload time on constrained field connectivity.
Frame Rate	30 fps	Standard for smooth egocentric walkthrough footage without inflating file size.
Codec	H.264 (AVC)	Universal hardware-encoder support across the Android device range this app targets, keeping battery/thermal impact low during a 10-minute continuous encode.
Target Bitrate	~8,000 kbps	Visually clean at 1080p30 while keeping a 10-minute chunk in the low hundreds of MB, not gigabytes.
Zoom Factor	0.5x or 0.6x, device-dependent (BR-02)	Selected once at session start via Chapter 5.1's capability ladder; fixed for the whole session, never changed mid-recording.
Camera	Rear-facing only (BR-01)	Egocentric, forward-facing viewpoint.
Audio	AAC, mono, 128 kbps	Voice/ambient audio alongside video; mono keeps size down since stereo separation isn't meaningful for a body-worn/handheld capture.

2. Device-Tier Adjustments

- On lower-end devices where the target bitrate would exceed the encoder's stable hardware capability, the pipeline steps down bitrate (never resolution or frame rate) in fixed increments — keeping the field of view and smoothness consistent across the Collector fleet, which matters more for usable training data (BR-02's own stated rationale) than a slightly larger file.
- Zoom factor selection (0.5x vs. 0.6x) is a fixed per-device-model decision made once at first launch and cached — not re-negotiated every session — so footage from the same device is always comparable to itself over time.

3. What This Chapter Defers

- How these parameters are actually fed into the OS encoder — Chapter 5.4 (Recording Pipeline).
- How the resulting file is checksummed and finalized — Chapter 5.5 (Video Processing).

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.3

Recording Lifecycle

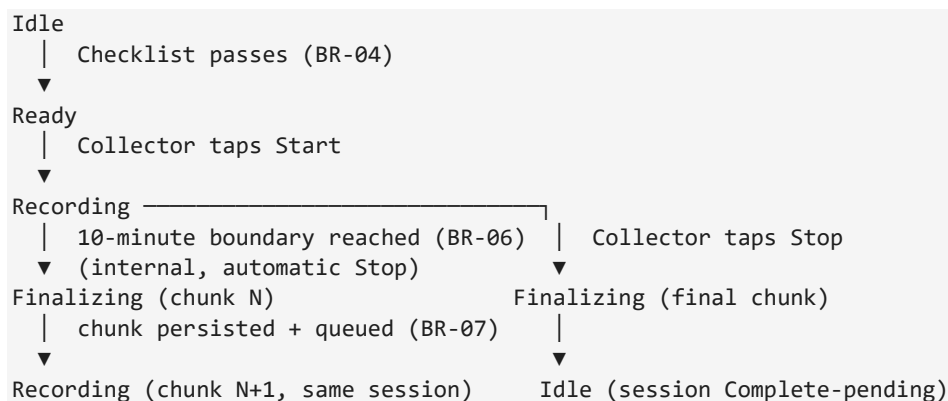
The State Machine Behind a Session — Including the Automatic 10-Minute Boundary

Document Title	Recording Lifecycle
Chapter Reference	5.3
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	BR-04, BR-05, BR-06 (Volume 1); RecordingNotifier (Volume 3, Chapter 3.9)

1. The Key Design Point — Chunking Is Always a Stop, Never a Slice

BR-05 is explicit: chunking happens only after the Collector taps Stop, never mid-recording. But the product's own promise is automatic 10-minute chunks (Volume 0, Vision Document) without the Collector doing anything. The lifecycle resolves this by treating the automatic 10-minute boundary as an internal, system-triggered Stop — identical in every way to a manual Stop, except it is immediately followed by a new Recording state rather than returning to Idle. This keeps BR-05's guarantee (no mid-stream slicing, which risks a corrupted encode) fully intact while still delivering seamless automatic chunking to the Collector.

2. State Diagram



3. States

Idle: no active session; the Collector is anywhere in the Task/Project navigation.

Ready: Checklist (C-07) has passed; camera is initialized (Chapter 5.1) but not yet recording.

Recording: actively capturing; a 10-minute internal timer runs alongside the Collector's ability to tap Stop at any time.

Finalizing: the brief Local Processing state (C-10) — video is finalized (Chapter 5.5), metadata is generated (Chapter 5.7), and the chunk enters the Upload Queue (Chapter 5.9). This is the same finalization path whether triggered automatically or manually.

4. Session vs. Chunk Boundaries

- A session begins when the Collector taps Start and ends only when the Collector explicitly ends it (returns to Task List) or the app determines the Task is complete — a session can and typically does span many automatic chunk boundaries.
- Every chunk within a session carries the same `session_id` and an incrementing `sequence_index` (Volume 4, Chapter 4.4) — the Recording Lifecycle is what actually increments that index, once per Finalizing transition.

- The final chunk of a session (whenever the Collector actually stops) may be shorter than 10 minutes (BR-06) — the lifecycle does not pad or discard a short final chunk.

5. Crash Recovery

If the app is killed mid-Recording, the in-progress chunk's partial file (Chapter 5.8's local storage layout) is either recovered and finalized on next launch (if enough of the encode is intact) or discarded per a minimum-duration threshold — never silently uploaded as if it were a complete, valid chunk. This is the lifecycle-level guarantee behind NFR-REL-04's "queue state fully restored on next launch."

6. What This Chapter Defers

- The actual encode-to-file mechanics — Chapter 5.4 (Recording Pipeline).
- What "finalized" concretely means for the video file — Chapter 5.5 (Video Processing).

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.4

Recording Pipeline

From Camera Frame to Bytes on Disk

Document Title	Recording Pipeline
Chapter Reference	5.4
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Pipeline Stages

Camera Module (5.1) → Hardware Encoder (5.2 params) → Muxer (video+audio → .mp4 container) → Buffered Writer → Local Filesystem (5.8)

- Camera Module streams raw frames at the fixed parameters from Chapter 5.2 to the device's hardware video encoder — never a software encoder, to keep battery/thermal load sustainable across back-to-back 10-minute sessions.
- The encoder's output and the audio track are muxed into a single .mp4 container in real time, not as a separate post-processing pass — this is what keeps Chapter 5.5's finalization step fast (NFR-META-01's < 500ms budget applies to metadata generation, not the mux itself, but a real-time mux is what makes that budget achievable).
- A buffered writer flushes to the device filesystem periodically (not only at Stop) so that a crash mid-chunk loses at most the last buffer interval, not the entire in-progress chunk — supporting the crash-recovery behavior named in Chapter 5.3.

2. Backpressure & Storage Checks

Pre-flight: the Pre-Recording Checklist (FR-CHK-02) already confirmed sufficient free storage before Recording state was entered (Chapter 5.3); the pipeline does not re-derive that decision.

Mid-recording: the buffered writer monitors remaining free space on every flush; if space becomes critically low mid-chunk, the pipeline forces an early chunk boundary (treated exactly like the automatic 10-minute boundary, Chapter 5.3) rather than allowing an out-of-space write failure to corrupt the active file.

3. Why Not a Single Continuous File, Chunked in Post

An alternative design would record one long file per session and slice it into 10-minute segments afterward. This was rejected: it would violate BR-07 (a chunk must be persisted to local storage before any upload of that chunk begins) for the entire session's duration, meaning nothing is safe on disk as an independently-recoverable unit until the whole session ends — directly contradicting the product's own core promise that a chunk uploads while the Collector keeps working. The chosen design (Chapter 5.3's internal-Stop pattern) produces a genuinely complete, independent file per chunk, the moment each boundary is reached.

4. What This Chapter Defers

- Container finalization and checksum computation — Chapter 5.5 (Video Processing).
- The on-disk path/naming for the file this pipeline writes — Chapter 5.8 (Local Storage) and Chapter 5.14 (File Naming Strategy).

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.5

Video Processing

Turning a Raw Recording Into a Finalized, Trustworthy Chunk File

Document Title	Video Processing
Chapter Reference	5.5
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	FR-META-06, FR-META-10 (Volume 1)

1. What Happens at a Chunk Boundary

- 1. Container finalization: the .mp4's moov atom (the index describing the file's tracks/duration) is written and the file handle is closed — before this step, the file is not a valid, playable video; the Recording Lifecycle (Chapter 5.3) does not consider a chunk 'finalized' until this completes.
- 2. Integrity checksum: a SHA-256 hash is computed over the finalized file, streamed in fixed-size blocks rather than loading the whole file into memory — this is FR-META-10's 'compute and store an integrity checksum for every chunk at creation time', and it is what Volume 4's backend later verifies against (FR-META-12).
- 3. File size capture: the final byte count is recorded (FR-META-06) directly from the closed file handle, never estimated.

2. Performance Budget

NFR-META-01 (Volume 1) caps metadata generation at under 500ms of added overhead per chunk. Checksum computation is the dominant cost here, so it is streamed and runs on a background isolate/thread separate from the UI, so the brief Local Processing screen (C-10) reflects genuine progress rather than blocking the main thread.

3. No Re-Encoding, No Thumbnail (MVP Scope)

No re-encoding: Chapter 5.4's real-time encode already produced the final file at the correct parameters (Chapter 5.2); this stage never re-transcodes, which would cost both time and a generation of quality loss for no benefit.

No thumbnail generation: not part of MVP scope (Volume 1, Chapter 1.10) — the Admin's Metadata Detail screen (A-08) shows metadata fields, not a video preview, in this phase; thumbnail extraction is a candidate for the Phase 3 QA/rework tooling named in Volume 1, Chapter 1.11, not resolved here.

4. What This Chapter Defers

- Where the finalized file and its checksum are recorded locally — Chapter 5.7 (Metadata Generation) and Chapter 5.8 (Local Storage).

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.6

Video Chunking

The Rules Behind Every Chunk Boundary

Document Title	Video Chunking
Chapter Reference	5.6
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	BR-05, BR-06, BR-07 (Volume 1)

1. Chunking Rules

Rule	Behavior
BR-05	A chunk boundary is always an internal or Collector-initiated Stop (Chapter 5.3) — never a slice of a continuous file.
BR-06	Every chunk is exactly 10 minutes, except the session's final chunk, which may be shorter.
BR-07	A chunk is written to local storage in full (Chapter 5.4/5.5) before it is ever handed to the Upload Queue (Chapter 5.9) — no chunk is ever uploaded 'live' while still recording.

2. Sequencing

- Each chunk is assigned `sequence_index` starting at 0 (or 1, matching Volume 4's convention) within its session, incremented once per Finalizing transition — this index is what lets both the mobile UI (C-11) and the Admin view (A-07) show chunks in recording order, independent of upload completion order (a later chunk may finish uploading before an earlier one that hit a retry).
- `session_id` is generated once, at session start, and stamped on every chunk in that session — chunking never creates a new `session_id`.

3. Edge Cases

Collector stops within seconds of a session starting: a very short final chunk is still valid and still fully processed (Chapter 5.5) — there is no minimum chunk duration below which footage is discarded, since even a short clip may be usable data.

Automatic boundary and manual Stop coincide: the automatic 10-minute timer is cancelled the instant a manual Stop begins finalization, so a chunk is never accidentally double-finalized.

App killed exactly at a boundary: resolved by Chapter 5.3's crash-recovery rule — an incomplete chunk is either recovered to a valid state or discarded, never left in an ambiguous, half-registered state in the Upload Queue.

4. What This Chapter Defers

- How a finalized chunk becomes a metadata record — Chapter 5.7.
- Exact file/object naming — Chapter 5.14.

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.7

Metadata Generation

The Mobile-Side Half of Volume 4's Metadata Specification

Document Title	Metadata Generation
Chapter Reference	5.7
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	FR-META-08/09/10 (Volume 1); the JSON shape defined in Volume 4, Chapter 4.5

1. Generation Trigger

Metadata generation begins the instant Chapter 5.5's Video Processing completes for a chunk — not lazily on upload (FR-META-08). The MetadataRepository (Volume 3, Chapter 3.4) assembles the exact JSON object specified in Volume 4, Chapter 4.5, populated as follows:

2. Field Sources

identity	← session/project/task/collector IDs already held by the active Recording Lifecycle state (Ch.5.3); device_id from a cached, stable device identifier.
timing	← sequence_index (Ch.5.6); started_at/ended_at timestamps captured by the Recording Pipeline (Ch.5.4).
capture	← the fixed parameters from Ch.5.2, plus the zoom factor actually selected by Ch.5.1's capability ladder.
device_context	← OS-reported device model/OS version; app_version from the build.
capture_conditions	← GPS (if permission granted), battery %, network type — read at the moment of chunk finalization, not session start.
integrity	← file_size_bytes and checksum_sha256 from Ch.5.5.
collector_authored	← empty at generation time; the only group a Collector may edit afterward (FR-META-07, BR-22).

3. Immediate Local Persistence (FR-META-09)

The assembled object is written to the local Drift chunk_metadata table (Chapter 5.8) in the same transaction as the chunk's own local record — both succeed or both fail together, so a chunk file can never exist locally without its metadata already alongside it, matching FR-META-09's 'persist immediately, surviving crash exactly as the video does'.

4. What Happens Next

Enters the Upload Queue: as a unit with its parent chunk (Chapter 5.9) — metadata and chunk travel together, per Volume 4's FR-META-11 upload-together rule.

Never re-generated: if a chunk's upload fails and is retried (Chapter 5.13), the exact same metadata object is resent — it is never recomputed, since capture_conditions like battery % and GPS are only meaningful as of the actual capture moment, not the retry moment.

5. What This Chapter Defers

- The local table schema this writes to — Chapter 5.8 (Local Storage).
- The backend-side verification step — Volume 4, Chapter 4.5 (already specified there).

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.8

Local Storage

The On-Device Mirror of Volume 4's Backend Schema

Document Title	Local Storage
Chapter Reference	5.8
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Engine	Drift (SQLite) — ADR-002, Volume 3, Chapter 3.2

1. Local Drift Tables

A deliberately small subset of Volume 4's backend schema — only what the device needs to function fully offline (Volume 0's offline-first principle) and to survive a crash (NFR-REL-04):

Table	Mirrors	Notes
local_sessions	sessions	Only this Collector's own sessions — never another Collector's.
local_chunks	chunks	Adds a <code>local_file_path</code> column with no backend equivalent.
local_chunk_metadata	chunk_metadata	Identical field shape to Volume 4, Chapter 4.5's JSON.
local_task_cache	tasks / task_assignments (read-only cache)	Refreshed from the API when online; lets C-03–C-06 render offline from last-synced data.

2. Filesystem Layout for Raw Video

```
<app-documents>/recordings/
  {session_id}/
    {sequence_index:04d}.mp4      # finalized chunk (Ch.5.5)
    {sequence_index:04d}.mp4.tmp  # in-progress write (Ch.5.4) – never
                                   # treated as a valid chunk until renamed
```

The `.tmp-suffix-until-renamed` pattern is the concrete mechanism behind Chapter 5.3's crash-recovery rule: on app relaunch, any `.tmp` file is either completed (if enough of the encode is salvageable) or deleted — a `.tmp` file is never picked up by the Upload Queue (Chapter 5.9) by construction, since the queue only ever watches for the renamed, extension-clean file.

3. Storage Quota Interaction

Pre-flight (FR-CHK-02): the Checklist already confirms free space before a session starts.

Ongoing: Chapter 5.4's backpressure check reads free space directly from the filesystem, not from a cached Drift value, since available space changes independently of anything this app writes (other apps, OS updates).

4. What This Chapter Defers

- The queue semantics built on top of `local_chunks.status` — Chapter 5.9 (Upload Queue).
- When `local_chunks` rows and their files are actually deleted — Chapter 5.15 (Storage Cleanup).

CHAPTER 5.9

Upload Queue

The Local, Persistent Model Behind Every Status Pill on C-11

Document Title	Upload Queue
Chapter Reference	5.9
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	FR-UPL-01/04/05 (Volume 1); StreamNotifier pattern (Volume 3, Chapter 3.9)

1. Queue States

Identical to the backend's `chunks.status` (Volume 4, Chapter 4.4) and the Chapter 2.8 status palette — the mobile queue and the backend never invent two different vocabularies for the same concept:

State	Meaning	Entered When
Queued	Waiting to upload	Chunk finalized (Ch.5.6/5.7); no connectivity, or ahead of other chunks in line.
Uploading	Actively transferring	Background Upload (Ch.5.11) has claimed this chunk and Ch.5.10's pipeline is running.
Failed	Attempt(s) exhausted for now	Ch.5.13's retry strategy gives up on the current attempt; surfaces the Retry Chunk action (C-11).
Complete	Backend-confirmed, checksum-verified	Volume 4, Chapter 4.5's <code>verified_at</code> is set and reflected back to the device.

2. Ordering

- FIFO by session start time, then by `sequence_index` within a session — an earlier session's chunks are not starved by a later session's, and within one session, chunks upload in recording order by default (though FR-UPL-04 means a later session can already be recording while an earlier one is still queued/uploading).
- A chunk that fails and is manually retried (FR-UPL-07) re-enters the queue at its original position, not pushed to the back — so retrying doesn't inadvertently delay every chunk behind it further.

3. Persistence

The queue is not an in-memory list — it is a live view (Riverpod StreamNotifier, Chapter 3.9) over `local_chunks.status` in Drift (Chapter 5.8). This is what satisfies NFR-REL-04 directly: an app kill loses nothing, because the queue's actual state was never only in memory to begin with. On relaunch, the queue simply resumes reading the same rows.

4. Manual Upload Mode (FR-UPL-02)

When a device is configured for manual upload, chunks still enter the queue in the Queued state immediately at finalization — the only difference is that Chapter 5.11's Background Upload dispatcher does not automatically claim them; it waits for an explicit Collector-triggered upload action instead. The queue's own model is identical in both modes.

5. What This Chapter Defers

- What actually moves a chunk from Queued to Uploading to Complete — Chapter 5.10 (Upload Pipeline) and Chapter 5.11 (Background Upload).
- Retry timing and give-up conditions — Chapter 5.13 (Retry Strategy).

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.10

Upload Pipeline

The Mobile-Side Steps of Volume 4's Multipart Upload Flow

Document Title	Upload Pipeline
Chapter Reference	5.10
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	FR-UPL-03/09 (Volume 1); the S3 flow specified in Volume 4, Chapter 4.10

1. Pipeline Steps

1. Register	POST /v1/sessions/{id}/chunks → presigned multipart URLs (Volume 4, Ch.4.6)
2. Upload	Dio (ADR-004) streams the local chunk file in parts directly to S3 using those URLs – the app never touches an AWS credential.
3. Confirm	PATCH /v1/chunks/{id}/status → 'uploading' → 'complete'
4. Metadata	POST /v1/chunks/{id}/metadata (Ch.5.7's object, sent as-is)
5. Verify	Backend checks checksum (Ch.4.5) → verified_at set → device's Upload Queue (Ch.5.9) observes chunks.status='complete' via its next sync and updates the local row to match.

2. Progress Reporting

Dio's native upload-progress callback (Volume 3, ADR-004) feeds directly into the per-chunk percentage shown on C-11 (Chapter 2.7) — no separate progress-tracking mechanism is built; the same callback that drives the network call also drives the UI.

3. Idempotency — Why a Retry Never Duplicates (FR-UPL-09, BR-11)

Same object key, every attempt: Step 1's registration always returns the same deterministic S3 key for a given chunk_id (Chapter 5.14) — a retried registration call is safe to repeat, and S3's own multipart upload semantics mean re-uploading a part with the same part number overwrites that part rather than duplicating it.

Confirm is idempotent: PATCH .../status to 'complete' is a no-op if the chunk is already Complete — the pipeline can safely re-send it after a dropped response without side effects.

4. Cancellation & Pause (FR-UPL-08)

Dio's cancellation tokens (ADR-004) allow an in-progress upload to pause cleanly mid-multipart-transfer; resuming re-uses the same upload ID and re-sends only the parts not yet acknowledged by S3, rather than restarting from byte zero — this is the mechanism behind NFR-REL-02's 'resumable after any interruption'.

5. What This Chapter Defers

- How this pipeline actually runs while the app is backgrounded — Chapter 5.11.
- What happens when a step above fails outright — Chapter 5.13 (Retry Strategy).

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.11

Background Upload

Making ADR-003 Concrete, Platform by Platform

Document Title	Background Upload
Chapter Reference	5.11
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Expands	ADR-003 (Volume 3, Chapter 3.2)

1. Android — Foreground Service

- A single, long-running foreground service starts the moment any chunk enters Uploading (Chapter 5.9) and stays alive as long as the queue has anything left to send — not restarted per chunk, to avoid repeated notification flicker.
- A persistent notification (Chapter 2.9's honest-signal principle) shows aggregate progress ("Uploading 2 of 5 chunks"), dismissible only once the queue is fully drained or every remaining item is Failed awaiting manual retry.
- The service holds a partial wake lock only while actively transferring bytes — not for its entire lifetime — to limit battery impact during any idle gaps between chunks.

2. iOS — Background URLSession

- Each chunk's multipart upload uses a background URLSession configuration, which iOS itself manages and can continue even if the app is suspended — the app does not attempt to keep itself alive artificially.
- On relaunch (iOS may terminate and relaunch the app to deliver a background completion event), the app's applicationDidFinishLaunching handler reconnects to the pending background session and resumes updating the Upload Queue (Chapter 5.9) from wherever the transfer left off.
- iOS's background execution time is not unlimited — this is the risk Volume 3 flagged as secondary-priority; the mitigation is that a chunk interrupted mid-transfer simply falls back to Queued/Failed and is picked up again the next time the app is foregrounded or another background opportunity is granted, never silently lost (BR-08 already guarantees the local file survives regardless).

3. Shared Behavior Across Both Platforms

Concern	Behavior
Multiple chunks ready at once	Uploaded with limited concurrency (a small fixed number in parallel, not all at once) to avoid saturating a constrained field connection and starving the Recording Pipeline (Ch.5.4) of CPU/network priority.
App brought to foreground mid-upload	No interruption — the same pipeline (Ch.5.10) continues; only the notification/session-management mechanics differ by platform.
Airplane mode / connectivity lost mid-upload	Handed to Chapter 5.12 (Offline Mode) and Chapter 5.13 (Retry Strategy) — this chapter only owns keeping the OS from killing the attempt outright while a connection exists.

4. What This Chapter Defers

- Connectivity-loss and reconnection behavior — Chapter 5.12.
- Retry timing after a failure — Chapter 5.13.

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.12

Offline Mode

The Engineering Mechanism Behind Chapter 2.9's Offline UX Guidance

Document Title	Offline Mode
Chapter Reference	5.12
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	FR-UPL-05 (Volume 1); NFR-AVL-02 (< 30s resume)

1. What Stays Unaffected by Connectivity

- Recording (Chapters 5.3–5.6) has zero network dependency — the Pre-Recording Checklist's network check (FR-CHK-04) only decides whether upload starts immediately or is queued, never whether recording is allowed.
- Chunk finalization, metadata generation, and local persistence (Chapters 5.5, 5.7, 5.8) all run fully offline, identically to the online case.

2. ConnectivityService

A dedicated platform service (Volume 3, Chapter 3.4) subscribes to the OS-level network reachability API and exposes a simple online/offline signal to the rest of the app — the Upload Queue (Chapter 5.9) and Background Upload dispatcher (Chapter 5.11) both listen to it, but neither polls it independently, avoiding duplicated battery cost.

3. Behavior While Offline

- Every finalized chunk still enters the queue as Queued (Chapter 5.9) — offline is not a blocked state, only a waiting one.
- The Collector can keep recording additional sessions without limit while offline (FR-UPL-04, US-18) — the queue simply grows; Chapter 5.8's storage-quota checks are what actually bound how long this can continue, not connectivity.

4. Resuming on Reconnection (NFR-AVL-02)

Signal: ConnectivityService fires an online transition the moment the OS reports connectivity restored.

Action: the Background Upload dispatcher (Chapter 5.11) immediately claims the front of the queue — no polling delay, no Collector action required — which is what keeps this comfortably inside NFR-AVL-02's 30-second target; the target is dominated by the OS's own reconnection detection latency, not by anything this app adds on top.

Visible confirmation: queued pills (Chapter 2.8) transition to uploading the moment this happens, giving the Collector the same 'it's automatically resuming' feedback specified in Chapter 2.9.

5. What This Chapter Defers

- What happens when an upload attempt itself fails even with connectivity present — Chapter 5.13 (Retry Strategy).

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.13

Retry Strategy

How Long the App Keeps Trying Before It Asks the Collector

Document Title	Retry Strategy
Chapter Reference	5.13
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	FR-UPL-06/07 (Volume 1); NFR-REL-02/03

1. Failure Classification

Failure Type	Examples	Handling
Transient	Connection dropped mid-transfer, S3 5xx, timeout	Automatic retry per Section 2's backoff — never surfaced to the Collector as Failed until attempts are exhausted.
Terminal (device-side)	Local file missing/corrupted, disk full	Not retried automatically — surfaces immediately as Failed with a specific, named cause (Chapter 2.9's copy rules).
Terminal (server-side)	Backend rejects with a 4xx (e.g. auth/permission error)	Not retried automatically — a repeated identical request would fail identically; surfaces as Failed.

2. Automatic Backoff (Transient Failures)

- Exponential backoff: 5s, 10s, 20s, 40s, capped at 5 minutes between attempts.
- Up to 6 automatic attempts per chunk before it transitions to Failed and waits for either connectivity/context to change (Chapter 5.12) or a manual retry (FR-UPL-07, C-11's Retry Chunk button).
- A small random jitter ($\pm 20\%$) is added to each delay so that many chunks failing at once (e.g. a whole batch losing connectivity together) don't all retry in the same instant and thundering-herd the backend the moment connectivity returns.

3. Manual Retry

Tapping Retry Chunk (C-11) resets the attempt counter and immediately tries again, regardless of how long the chunk has been Failed — this is a deliberate Collector override of the automatic backoff schedule, not a bypass of Section 1's terminal-failure classification (a terminal failure's underlying cause, e.g. a permission error, will simply fail again immediately, still with its specific named reason shown).

4. Duplicate Prevention Under Retry (FR-UPL-09, BR-11)

Every retry — automatic or manual — reuses the exact same chunk_id, the exact same deterministic S3 key (Chapter 5.14), and where possible the same in-progress multipart upload ID (Chapter 5.10). This is what makes 'retry' safe by construction rather than by discipline: there is structurally no code path that generates a new key for the same chunk.

5. What This Chapter Defers

- The exact deterministic key retries rely on — Chapter 5.14 (File Naming Strategy).

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.14

File Naming Strategy

The Deterministic, Collision-Proof Pattern Named in Volume 0's Constitution

Document Title	File Naming Strategy
Chapter Reference	5.14
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Resolves	Volume 0, Ch.0.2 — "S3 object keys / chunk files: deterministic, collision-proof pattern finalized in Volume 5.14"; referenced in Volume 4, Chapter 4.10.

1. S3 Object Key (Authoritative Definition)

```
{org_id}/{project_id}/{task_id}/{session_id}/{sequence_index:04d}_{chunk_id}.mp4
```

Example:

```
org_9f2/proj_4a1/task_7c3/sess_e810/0003_chk_5b2a.mp4
```

This is the same pattern shown as an example in Volume 4, Chapter 4.10 — this chapter is its authoritative source, since it must be computed identically on the mobile client (for local file naming, Section 2) and the backend (for the S3 key it hands back via presigned URL, Volume 4 Chapter 4.6).

2. Local Filesystem Naming

```
<app-documents>/recordings/{session_id}/{sequence_index:04d}.mp4
```

```
<app-documents>/recordings/{session_id}/{sequence_index:04d}.mp4.tmp (in-progress)
```

The local path omits org/project/task since those are already implied by the session (a lookup, not a duplication) and the local filesystem doesn't need global uniqueness across every Collector's device — only global uniqueness within one device's own storage, which session_id + sequence_index already guarantees.

3. Why This Pattern Is Collision-Proof

chunk_id: a UUID generated locally the moment a chunk begins finalizing (Chapter 5.6) — never reused, never predictable, never assigned by a counter that could collide across devices.

session_id: a UUID generated once at session start (Chapter 5.3) — two Collectors, or the same Collector on two devices, can never produce the same session_id.

sequence_index: scoped to its own session_id, so it only needs to be unique within one session, which the Recording Lifecycle already guarantees by incrementing it exactly once per chunk boundary (Chapter 5.6).

Result: the full key is unique by construction, without any coordination with the backend or any other device — and because it is fully deterministic from data the app already has (never randomly regenerated), a retried upload (Chapter 5.13) always recomputes the identical key, which is precisely what makes BR-11's 'never a duplicate object in S3' guarantee hold.

4. Human-Readability Trade-off

The key is intentionally ID-based rather than name-based (e.g. not using the Project's display name) — display names can change (an Admin renaming a Project, FR-ADM-02) and must never invalidate or orphan already-uploaded footage. Any human-readable presentation (e.g. in an Admin export, A-08) is generated by joining these IDs back to their current display names at read time, not baked into the storage key itself.

5. What This Chapter Defers

- What happens to a file at this path once it's no longer needed locally — Chapter 5.15 (Storage Cleanup).

VOLUME 5 — RECORDING ENGINE

CHAPTER 5.15

Storage Cleanup

Reclaiming Local Disk Space Without Ever Risking Data Loss

Document Title	Storage Cleanup
Chapter Reference	5.15
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Implements	BR-08 (Volume 1); FR-META-14, BR-23 (Volume 1)

1. The One Rule Everything Else Follows

BR-08: a chunk shall not be deleted from local storage until its upload to S3 is confirmed. Every mechanism in this chapter exists only to decide when that condition is safely true — never to work around it.

2. Cleanup Trigger

- A chunk becomes eligible for local deletion the instant the Upload Queue (Chapter 5.9) observes its status reach Complete — i.e. the backend has confirmed both the S3 object and its metadata (Volume 4, Chapter 4.5's `verified_at`).
- Cleanup itself runs as a low-priority background sweep, not synchronously at the moment of eligibility — deleting a large video file is not time-critical, and batching it avoids competing with an active Recording Pipeline (Chapter 5.4) for I/O.
- Only the raw video file (and its `local_chunks` row's file reference) is removed. The `local_chunk_metadata` row is never deleted by this sweep — see Section 3.

3. Metadata Outlives the File (FR-META-14, BR-23)

Local metadata rows remain queryable after their video file is cleaned up, since the backend — not the device — is the durable, canonical copy an Admin actually queries (FR-ADM-06); the local Drift copy is retained defensively but is not the system of record. If local storage pressure ever required trimming old, already-synced metadata rows too, that would be a data-retention-policy decision (Volume 8.7), not something this chapter authorizes on its own.

4. Storage Pressure & Backpressure

Normal operation: the eligibility rule in Section 2 alone usually keeps local storage bounded, since chunks are actively cycling through the queue rather than accumulating.

Sustained offline period: if the Collector is offline long enough that many chunks accumulate (Chapter 5.12), none of them are eligible for cleanup yet (BR-08 still applies) — this is exactly why the Pre-Recording Checklist's storage check (FR-CHK-02) exists: it is the honest signal that the device is genuinely running low, at which point the Collector is blocked from starting a new session (not silently allowed to overwrite or skip BR-08) until either connectivity returns or space is freed some other way (e.g. manual file management).

5. What This Chapter Deliberately Does Not Do

- It never deletes a chunk based on age alone, disk pressure alone, or any heuristic that isn't 'backend-confirmed Complete' — BR-08 has no exception clause, and this document introduces none.
- Long-term backend-side retention limits (how long S3/metadata are kept before eventual purge) are Volume 8.7's policy decision, not a mobile-engine concern.